Operating System Fundamentals - EECS3221
Project 1
Simulating CPU Scheduling Algorithms
Prepared by: Dr. Ruba Al Omari[1]

Contents

Instructions 1
Description 1
Implementation 2
Tasks Storage and Organization 3
Tiebreaking Rules 3
StarterKit 3
Evaluating Your Implementation 4
Code Documentation Requirements 5
Deliverables 5
Rubric 7
Collaboration 8
Generative AI Policy 8
Environment Setup 9

Instructions

- This is an individual project.
- Check eClass for the submission due date and time.

Description

This project involves implementing several different process scheduling algorithms. The scheduler will be assigned a predefined set of tasks and will schedule the tasks based on the selected scheduling algorithm. Each task is assigned a priority and CPU burst. The following scheduling algorithms will be implemented:

- **First-Come, First-Served (FCFS):** Tasks are scheduled in the order they request the CPU.
- **Shortest Job First (SJF):** Tasks are scheduled in order of the length of their next CPU burst.
- **Priority Scheduling:** Tasks are scheduled based on their priority.
- **Round-Robin (RR):** Tasks are scheduled in a cyclic order, with each task running for a fixed time quantum (or the remainder of its CPU burst if less than the quantum).

[1] This project is provided as a programming project in Operating System Concepts, 10th Edition, by Silberschatz et al. textbook.

- **Priority with Round-Robin:** Tasks are scheduled based on priority. For tasks with the same priority, round-robin scheduling is applied using the defined time quantum.

The following metrics will be calculated:

1. **Waiting Time:** Total time spent waiting in the ready queue
2. **Turnaround Time:** Time from arrival to completion
3. **Response Time:** Time from arrival to first execution start

Priorities range from 1 to 10, with higher numeric values representing higher priorities.
For RR scheduling, the time **quantum** is fixed at **10** milliseconds.
For Priority with Round-Robin scheduling, the time **quantum** is also fixed at **10** milliseconds for tasks with the same priority.

The goal of this project is to demonstrate the behaviour and performance of these scheduling algorithms under different conditions.
Scheduling algorithms are discussed in Section 5.3 in the course textbook (numbering may vary slightly by edition).

Implementation

The implementation for this project must be completed in C, with supporting program files provided in the `StarterKit` folder. These supporting files read in the schedule of tasks, insert the tasks into a list, and invoke the scheduler.

The schedule of tasks has the form `[task name][priority][CPU burst]`, with the following example format:

```
T1, 4, 20
T2, 2, 25
T3, 3, 25
T4, 3, 15
T5, 10, 10
```

In this example:

- Task **T1** has a priority of **4** and a CPU burst of **20** milliseconds.
- Task **T2** has a priority of **2** and a CPU burst of **25** milliseconds, and so on.

It is assumed that all tasks arrive at time 0. **Note:** None of the implemented algorithms should involve preemption except where naturally required by the algorithm (e.g., RR and Priority_RR inherently use time slicing). Priority and SJF should be non-preemptive implementations.
Input Validation: You may assume valid input (priority is an integer from 1–10, and burst > 0)

Tasks Storage and Organization

There are a few different strategies for organizing the list of tasks:

- Single Unordered List: One approach is to place all tasks in a single unordered list, where the strategy for task selection depends on the scheduling algorithm. For example, Shortest Job First (SJF) would search the list for the task with the shortest CPU burst.
- Ordered List: Alternatively, a list could be ordered according to scheduling criteria (that is, by priority).
- Separate Queues for Each Priority: One other strategy involves having a separate queue for each unique priority, as shown in Figure 5.7.

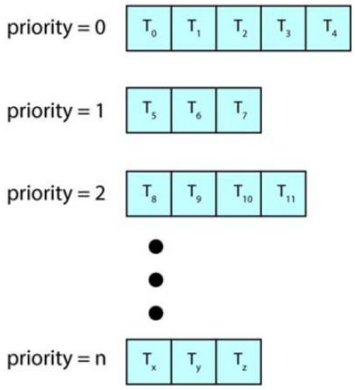


Figure 5.7 Separate queues for each priority.

It is also worth highlighting that we are using the terms **list** and **queue** somewhat interchangeably. However, a queue has very specific FIFO functionality, whereas a list does not have such strict insertion and deletion requirements. You are likely to find the functionality of a general list to be more suitable when completing this project.

Tiebreaking Rules

When scheduling tasks:

- Processes must execute in the order they appear in the input if there are ties in priority or burst time.
- This rule applies to ties during SJF, Priority Scheduling, or Priority with Round-Robin.

StarterKit

The file `driver.c` reads in the schedule of tasks, inserts each task into a linked list, and invokes the process scheduler by calling the `schedule()` function.

The `schedule()` function executes each task according to the specified scheduling algorithm.

As part of your scheduler, you must determine which task should run next. For example, you might implement a `pickNextTask()` function to choose a task, and then use the `run()` function defined in `CPU.c` to simulate its execution. The `run()` function is provided; the task-selection logic is your responsibility.

A `Makefile` is used to determine the specific scheduling algorithm that will be invoked by the driver. For example, to build the FCFS scheduler, we would enter:

```
make fcfs
```

and would execute the scheduler (using the schedule of tasks schedule.txt) as follows:

```
./fcfs schedule.txt
```

Before proceeding, be sure to familiarize yourself with the source code provided as well as the `Makefile`. Here is a quick tutorial on `Makefile` http://www.cs.colby.edu/maxwell/courses/tutorials/maketutor/ in case you are not familiar with it.

Evaluating Your Implementation

- The correct outputs for each algorithm are provided in separate files (e.g., `correct_fcfs-output.txt, correct_sjf-output.txt,` etc.). Compare your program's output with these reference files.

- Your program must print each task execution using the following exact format:
  `Running task = [TASK_NAME] [PRIORITY] [REMAINING_BURST] for [TIME_SLICE] units`

- And finally display averages:
  `Average waiting time = [VALUE]`
  `Average turnaround time = [VALUE]`
  `Average response time = [VALUE]`

- To evaluate whether your code is running correctly:
  Step 1: Run your code, and manually open each of the output files produced by your code and verify that the files are not empty.
  Step 2: Use the `grader_student.sh` script to check that your results are correct. Ensure your implementation works with any valid input file, not just `schedule.txt`. TAs have a different script with different file names.
  Note: Before running the `grader_student.sh`, verify that your output files are not empty. An empty output file may prevent the grader from detecting errors correctly.
  If you encounter permission errors when running the provided `grader_student.sh` script, make sure to troubleshoot and resolve them before submission, this may involve adjusting file permissions or verifying script syntax.

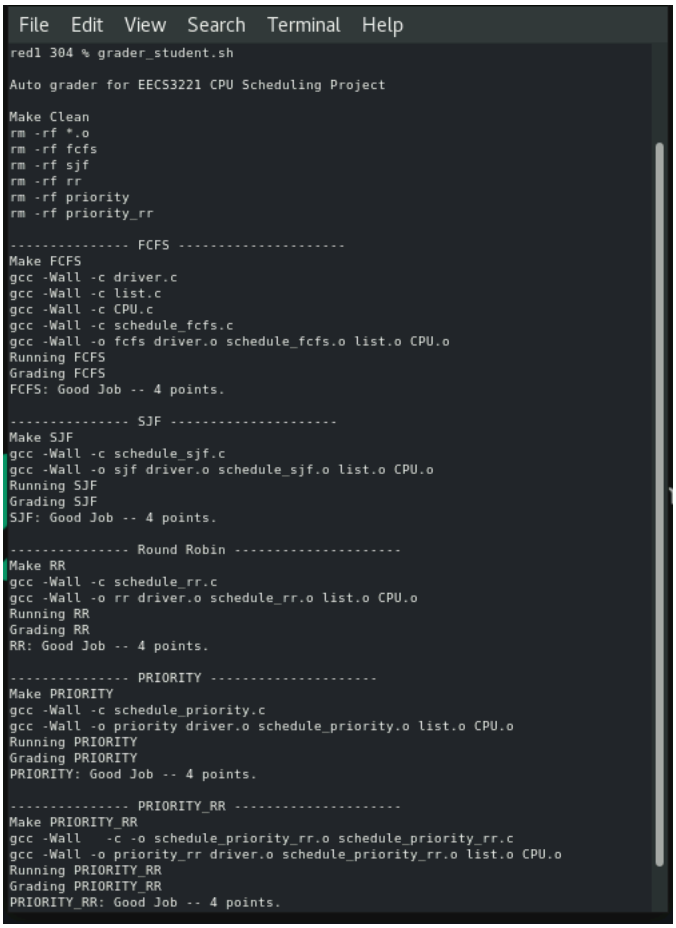


Figure 1. Example output from running `grader_student.sh` successfully, showing all scheduling algorithms passing with full points.

Note: It is your responsibility to troubleshoot and resolve any issues that affect the grader, including but not limited to extra blank lines, formatting differences, incorrect file names, missing files, permission problems, or other errors that may cause grader_student.sh to produce unexpected results or fail to evaluate your submission correctly.

Code Documentation Requirements

- Each C file must begin with a comment header containing:
  1. Your name and student ID
  2. Brief description of the file's purpose
  3. If generative AI was used, include a brief statement describing how it was used (e.g., debugging assistance, pseudocode suggestions, explanation of concepts).
- Key functions should have brief comments explaining their logic
- Complex sections of code should include explanatory comments

Deliverables

Completing this project will require that you:

1. Write the following C files, which invoke the appropriate scheduling algorithm:
    1. `schedule_fcfs.c`
    2. `schedule_sjf.c`
    3. `schedule_rr.c`
    4. `schedule_priority.c`
    5. `schedule_priority_rr.c`
2. Calculate the *average* **turnaround time**, **waiting time**, and **response time** for each of the above scheduling algorithms.
3. Submit one zip file, called `Yourfirstname_cpu.zip`, that includes:
    - The `StarterKit` (even if it was not modified)
    - The `Makefile` (do not change the provided `Makefile`)
    - Any source files that were added
    - **Do not** submit object (*.o) files or compiled executables
4. Submit one video file that demonstrates your program working correctly:
    1. Log in to RemoteLab.
    2. Unzip your submitted `Yourfirstname_cpu.zip` file on RemoteLab.
    3. Run the `grader_student.sh` script showing the output.
    4. Explain **two** aspects of your code (see the Rubric section for more details).
    5. Your recording should be **6-8 minutes**.
    6. Your recording must be uploaded to eClass as an MP4 file (do not submit links).
5. Additional requirements:
    - Ensure your project runs correctly on the Department's servers with the required configuration (see Environment Setup).
    - TAs will compile your submission using the provided `Makefile`; do **not** change the content of the provided `Makefile`.
    - Grading will be performed using input files with different names. Your algorithms must not be hard-coded to `schedule.txt`; they should work with any input file name.
    - Your implementation must work for any valid input file following the same format. Do not hardcode for specific task counts, names, or values.
    - Test your code using the file `grader_student.sh` provided in the StarterKit.

Submission Checklist:

- ☐ All **5** scheduling algorithm files completed
- ☐ `Makefile` unchanged from provided version
- ☐ All source files include proper header comments
- ☐ AI assistance is properly documented if used
- ☐ Code compiles without warnings on EECS servers
- ☐ Self-grade script produces correct output
- ☐ Video recording (6-8 minutes) in MP4 format
- ☐ Single ZIP file including the code named: Yourfirstname_cpu.zip.

Rubric

A. **Scheduling Algorithm Implementation (20 points total, 4 points each)**

For each of the 5 scheduling algorithms:

1. **Correct Output** (Order and Values): 2.5 points.
2. Deductions for errors in the output:
   - One incorrect line: -0.5 point
   - Two incorrect lines: -1 point
   - Duplicate lines: -0.5 point
   - Three or more incorrect lines: -2.5 points (no credit for this section)
3. **Correct Average Waiting Time**: 0.5 point
4. **Correct Average Turnaround Time**: 0.5 point
5. **Correct Average Response Time**: 0.5 point

An *incorrect line* refers to any line in the output that shows incorrect: (1) task execution order, (2) execution duration, or (3) timing calculations.
Duplicate lines refer to printing the same task execution line multiple times in the output when it should only appear once.

B. Video Demonstration (5 points)

Record your computer screen:

1. Unzip your `Yourfirstname_cpu.zip` file on RemoteLab.
2. Run the `grader_student.sh` script showing the output.
3. Choose **TWO** of these aspects to explain (pick what you're most proud of or found most challenging), and open your source .c file to explain your code for each chosen aspect:

   A. Data Structure Choice
   - Show how you store/organize tasks (linked list, array, queue, etc.)
   - Explain why you chose this structure
   - Show relevant code sections

   B. Algorithm-Specific Logic
   - **RR/Priority_RR:** How you handle quantum and remaining burst
   - **SJF/Priority:** How you select the next task and handle ties
   - **Time Calculations:** How you compute turnaround/waiting/response times

   C. Key Function Implementation
   - Pick one key function (e.g., `pickNextTask()`)
   - Walk through its parameters, logic, and return values
   - Show how it integrates with the overall scheduler

   D. Challenge You Solved
   - Describe a specific bug or challenge you encountered
   - Show your debugging approach and solution
   - Explain what you learned from it

Example Explanation (for RR algorithm):
"For my Round-Robin implementation, the key aspect was handling tasks with remaining burst less than the quantum. In my code (show code on the screen and explain as you highlight lines of code), you can see I check if `remaining_burst` < quantum. If true, I run the task for `remaining_burst` instead of quantum. This ensures tasks are completed correctly."
What NOT to do:

- Don't explain generic scheduling concepts (explain YOUR implementation)
- Don't read from the project description
- Don't show your entire code file without focusing on specific parts
- Don't exceed 8 minutes

Grading Focus: We're assessing that you understand YOUR code, not that you can recite definitions.

Total: 25 points (will be scaled to 20 points for your final grade)

Collaboration

Here are the allowed activities while working on this project:

1. **Discussing Problem-Solving Approaches**: You may discuss strategies for solving a problem. However, to preserve the learning experience of others, avoid revealing key solutions directly and instead guide them toward discovering the solutions themselves.
2. **Syntax and Bug Discussions**: You can discuss syntax issues and bugs in your code, provided you do not show your code to others. For example, verbally discussing issues is allowed, but screen sharing via tools like Zoom is not.

While working on this project, it is not allowed to be:

1. **Viewing Another Student's Code**: You may not view another student's project code to understand an idea or solve a problem, nor share your own code for this purpose.
2. **Possessing Unauthorized Code**: You may not possess project solution code you didn't write yourself, nor another student's code, in any form (electronic, physical, etc.), even for debugging purposes. Sharing such code is equally prohibited.

Generative AI Policy

You may use generative AI (ChatGPT, Copilot, etc.) to explain concepts, help debug, suggest high-level structure or pseudocode, and improve documentation. You may not copy-paste complete algorithm implementations, use AI to bypass the core algorithmic work, or submit AI-generated code with only superficial changes (renamed variables, reformatting, etc.).

You must be able to explain every line of your submitted code. This is assessed through the required video demonstration; if a TA has reason to doubt your understanding from the video or your submission, you may be asked to complete a short follow-up interview, tracing through your code with a new sample input.

Submitting AI-generated code without understanding it or without properly documenting AI assistance constitutes academic dishonesty.

Environment Setup

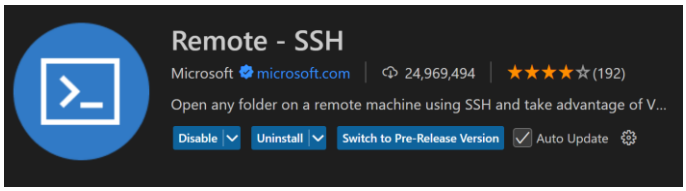
This section provides guidance for setting up EECS servers for remote development in C. You will need to configure the server and set up a client IDE (e.g., Visual Studio Code) to use the server as a backend for development.
You can use any EECS department server (e.g., red, red1, crimson, etc.) to write your code.

There are several ways you can use the servers:
Option 1: Physically visit one of the EECS labs, log in locally to the machine and start developing your code.

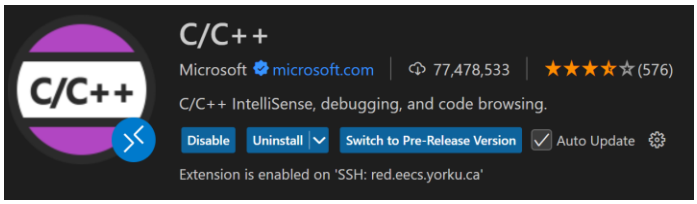
Option 2: If you can't be in the lab, you can still log in remotely to https://remotelab.eecs.yorku.ca and start developing your code.

Option 3: You can set up your local IDE to connect to the remote server following the steps below:

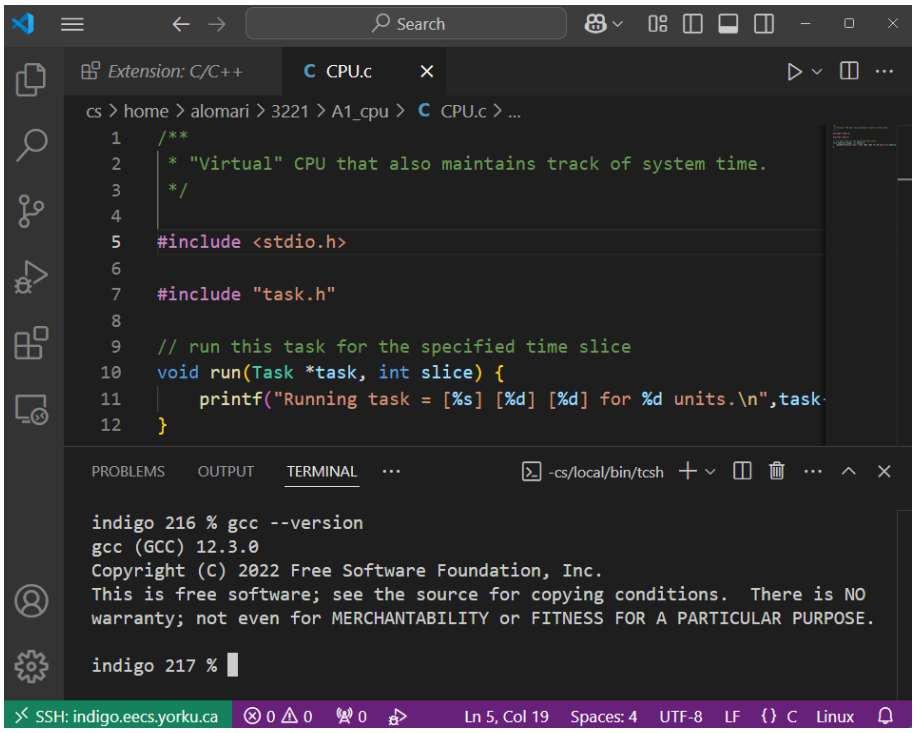
1. Download and install Visual Studio Code on your local machine.
2. Install Required Extensions:
   - Remote - SSH Extension: Allows remote connections to servers.



   - C/C++ Extension: Enables C/C++ development.



3. Create a remote connection to your, e.g., red1, server.
4. Open the terminal inside VSCode and verify the versions of GCC and make.



You are all set to start the development of your project code.

Note: GCC and other software versions on EECS servers may differ from those shown in the screenshot, as the systems are periodically updated. Your code should compile and run correctly on the current server environment.

If you encounter login issues, contact EECS IT support.